Jogo da Memória para Idosos

Aluno: Vitor Hugo do Prado **RA:** 2618109

Aluno: Eduardo Shioga **RA:** 2565579

Aluno: Arthur Simão Gimenes Dalio Pereira **RA:** 2565544

Aluno: João Vitor Taques Marques **RA:** 2648156

Aluno: João Vitor de Menezes Aguiar **RA:** 2649349

a) Descrição do Sistema Proposto (Valor: 0,5)

Nome do projeto: Jogo da Memória para Idosos

O projeto propõe um jogo da memória para apoiar atividades de inclusão digital do TEDI na UTFPR Cornélio Procópio. A atividade permite praticar a interação com uma interface digital por meio da escolha e da comparação de cartas com nomes.

A partida possui dois jogadores no mesmo tabuleiro de quatro linhas e quatro colunas, com dezesseis cartas organizadas em oito pares. O jogador seleciona duas cartas: ao acertar, recebe um ponto e continua; ao errar, as cartas são ocultadas e a vez passa ao outro participante.

O sistema acompanha pontos e tentativas e encerra a partida quando os oito pares são encontrados. Vence quem tiver mais pares. O resultado é empate se cada jogador encontrar quatro pares. Uma nova partida reinicia cartas, turnos, pontuação e tentativas.

O escopo desta etapa abrange cinco requisitos funcionais implementados em C. A página de testes demonstra a lógica compilada. Os dados permanecem na memória durante a partida, sem cadastro, autenticação ou banco de dados.

b) Definição das Ferramentas Utilizadas (incluindo a linguagem de programação) (Valor: 1,0)

O desenvolvimento utiliza as ferramentas descritas a seguir para implementar a lógica do jogo, compilar o código C e disponibilizar o protótipo no navegador.

Ferramenta / Tecnologia	Aplicação no projeto
C	Lógica do jogo.
Emscripten 6.0.11	Compila o C para WebAssembly e gera o JavaScript de suporte.
WebAssembly	Executa o código C compilado no navegador.
JavaScript, HTML e CSS	Criam o tabuleiro, recebem cliques e chamam as funções C.
Git e GitHub	Versionamento do código, colaboração e armazenamento do script.
GitHub Pages	Publicação dos arquivos estáticos da pasta docs/.
Python	Servidor HTTP local utilizado somente para testar os arquivos compilados.

c) Planejamento Inicial das Tarefas (Cronograma de Desenvolvimento com Divisão das Tarefas) (Valor: 1,0)

O planejamento do Jogo da Memória para Idosos foi organizado em cinco requisitos funcionais, com um responsável por módulo e integração final compartilhada. O registro abaixo distingue os marcos identificados nas mensagens do grupo das atividades ainda previstas para a entrega de 16 de outubro de 2026, às 18h40.

Divisão das tarefas

Vitor Hugo do Prado — RF01: criação e embaralhamento do tabuleiro; descrição do projeto e requisitos funcionais.

Arthur Simão Gimenes Dalio Pereira — RF02: seleção e comparação de duas cartas; ferramentas e tecnologias e diagrama de classes.

Eduardo Shioga — RF03: controle de turnos entre dois jogadores; planejamento, cronograma e slide do cronograma.

João Vitor Taques Marques — RF04: pontuação e tentativas; diagrama de casos de uso.

João Vitor de Menezes Aguiar — RF05: encerramento e resultado da partida; integração, compilação e publicação.

Organização do trabalho

O grupo utiliza o WhatsApp para combinar tarefas e compartilhar arquivos, enquanto o GitHub centraliza o código e a documentação. Foram previstas chamadas de alinhamento às quintas-feiras à noite. Os cinco módulos foram integrados e verificados por testes de integração.

Cronograma de desenvolvimento

Situação registrada em 15/10/2026. A versão consolidada foi testada localmente; a revisão da equipe e a publicação permanecem previstas.

Período	Atividade	Responsável	Situação
31/08 a 03/09	Definição do tema, equipe e proposta inicial	Equipe; envio por Vitor	Envio inicial registrado
13/09	Divisão dos cinco requisitos funcionais	Equipe	Responsáveis definidos
23/09	Disponibilização do código base do tabuleiro	Vitor	Base compartilhada
08/10	Criação do repositório e preparação da versão web	Menezes	Repositório disponível
15/10	Versão integrada dos cinco requisitos e testes locais	Equipe	Testes locais aprovados
15 e 16/10	Revisão dos módulos e publicação do jogo	Equipe; coordenação de Menezes	Previsto
16/10	Teste no computador da apresentação e ensaio	Equipe	Previsto
16/10 até 18h40	Envio do PDF no Moodle e apresentação presencial	Equipe	Entrega prevista

d) Levantamento dos Requisitos Funcionais (Valor: 1,0)

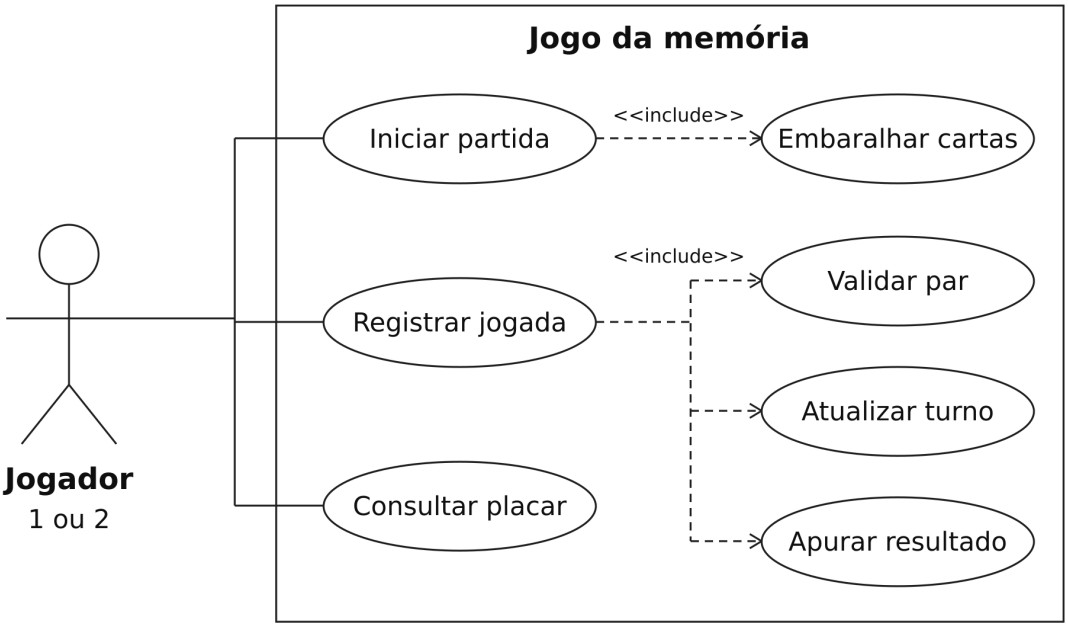
Abaixo estão descritos os Requisitos Funcionais (RF):

Código	Nome do Requisito	Descrição Detalhada	Prioridade
RF-001	Inicialização do Tabuleiro e Embaralhamento das Cartas.	O sistema cria um tabuleiro 4x4 e embaralha os nomes.	Alta
RF-002	Processamento da Jogada e Validação de Pares.	O sistema permite selecionar duas cartas e verifica se possuem nomes iguais, formando um par.	Alta
RF-003	Gerenciamento e Alternância de Turnos entre Jogadores.	O sistema mantém a vez do jogador quando ele encontra um par e passa a vez ao outro jogador quando ocorre um erro.	Alta
RF-004	Cálculo de Pontuação e Contagem de Tentativas.	O sistema registra uma tentativa por seleção completa de duas cartas válidas e soma um ponto por par encontrado.	Média
RF-005	Verificação do Fim do Jogo, Apuração	O sistema encerra a partida após os oito	Alta

Código	Nome do Requisito	Descrição Detalhada	Prioridade
	do Vencedor.	pares encontrados e identifica o jogador com maior pontuação, ou empate em 4 a 4.	

e) Diagrama de Casos de Uso (Valor: 1,5)

O ator Jogador representa os dois participantes, que utilizam o sistema na sua vez. O diagrama apresenta as ações de iniciar a partida, registrar uma jogada e consultar o placar, além das operações executadas pelo sistema.



A validação atualiza pontos e tentativas. Erro alterna a vez. Oitavo par encerra.

Figura 1 Casos de uso do jogo da memória

Fluxo principal da jogada

Ao selecionar duas cartas válidas, o sistema verifica se formam um par e registra uma tentativa. Um acerto soma um ponto e mantém a vez do jogador. Um erro passa a vez ao outro jogador e mantém as cartas visíveis até a chamada da função de ocultação. Enquanto houver um erro aguardando ocultação, novas seleções ficam bloqueadas. Ao encontrar os oito pares, o sistema permite consultar o vencedor ou empate e bloqueia novas jogadas.

f) Diagrama de Classes (Valor: 1,5)

O diagrama representa as estruturas de dados efetivamente utilizadas na versão integrada em C. As caixas mostram nomes e atributos das structs; suas operações são funções externas, pois C não possui métodos de classe.

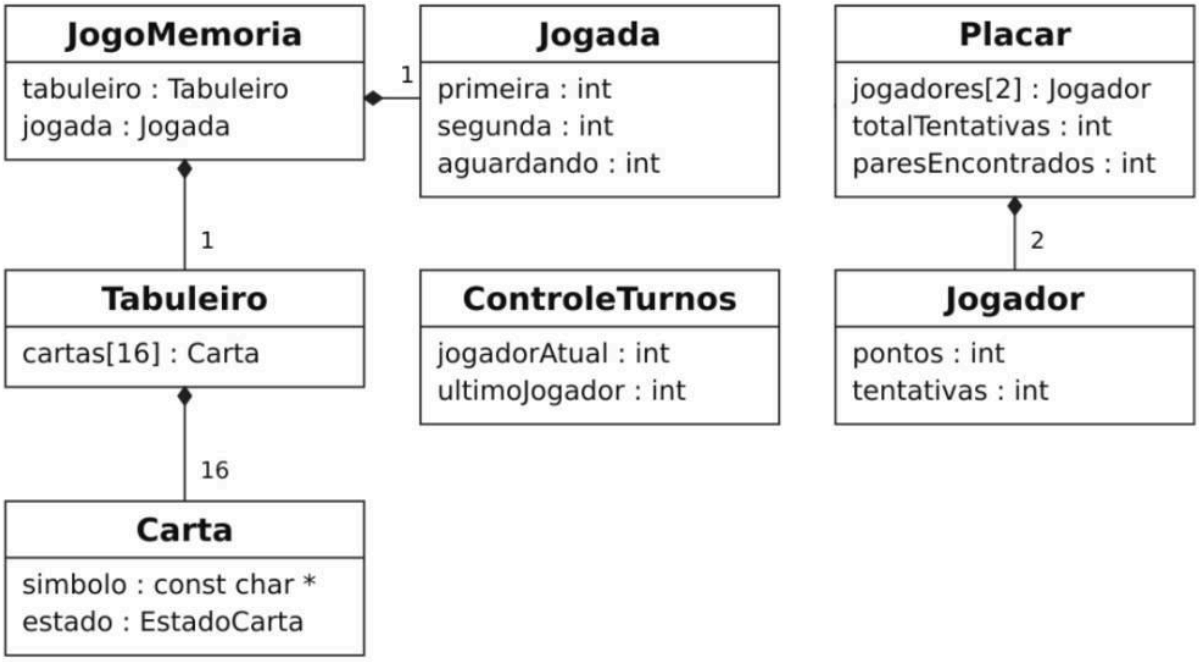


Figura 2 Classes conceituais e estruturas da implementação em C

Entidades e relacionamentos

JogoMemoria contém um Tabuleiro e uma Jogada. Tabuleiro contém dezesseis objetos Carta. Cada Carta armazena seu símbolo e seu estado. Jogada guarda as duas posições selecionadas e informa se é necessário aguardar a ocultação de um erro.

ControleTurnos guarda o jogador atual e o autor da última tentativa completa. Placar contém dois registros Jogador, o total de tentativas e a quantidade de pares encontrados. Cada Jogador possui pontos e tentativas.

O módulo resultado.c verifica o encerramento pela situação das cartas e apura o vencedor consultando a pontuação dos jogadores. Os módulos se comunicam pelas funções em projetocertificador.c, sem banco de dados.

g) Implementação (Codificação) do Back-End - 5 Funcionalidades (Valor: 3,5)

A versão consolidada implementa os cinco requisitos em C e disponibiliza uma interface de demonstração em HTML, CSS e JavaScript. O Emscripten 6.0.11 compila os fontes para WebAssembly. As regras da partida permanecem no código C.

Organização dos módulos

RF01 - Inicialização e embaralhamento: Embaralhar, em src/projetocertificador.c, reorganiza as dezesseis cartas e inicializa os estados da partida.

RF02 - Processamento da jogada: src/rf02.c e src/rf02.h validam as posições, comparam os símbolos, mantêm pares encontrados e permitem ocultar os erros.

RF03 - Gerenciamento de turnos: src/turnos.c e src/turnos.h mantêm a vez no acerto e alternam os jogadores no erro. O jogador 1 inicia uma nova partida.

RF04 - Pontuação e tentativas: src/pontuacao.c e src/pontuacao.h registram uma tentativa por par escolhido e somam um ponto no acerto, sempre para o jogador que realizou a tentativa.

RF05 - Encerramento e vencedor: src/resultado.c e src/resultado.h encerram a partida no oitavo par e comparam os pontos. O retorno distingue partida não iniciada, andamento, empate e vencedor.

Fluxo de integração

Ao concluir uma tentativa, a ponte C identifica o autor da jogada, atualiza a pontuação e consulta o encerramento. A tentativa e a atualização do turno ocorrem após a seleção da segunda carta válida, antes da ocultação. As funções em C disponibilizam a consulta dos estados da partida para a futura integração completa com a interface JavaScript.

Trechos centrais da implementação

Os trechos seguintes correspondem às funções utilizadas nos testes. O código completo, os cabeçalhos, os testes e os arquivos de compilação acompanham o pacote do projeto nas pastas: Os fontes estão em `src`, os testes em `tests` e os scripts de compilação na raiz do repositório. .

A interface visual é um protótipo. Os cinco requisitos funcionais são demonstrados pelos testes de integração do código C compilado para WebAssembly.

RF01 Inicialização e embaralhamento

```
EMSCRIPTEN_KEEPALIVE
void Embaralhar(void)
{
    const char *simbolos[RF02_TOTAL_CARTAS];

    if (!sementeDefinida) {
        srand((unsigned)time(NULL));
        sementeDefinida = 1;
    }
    for (int i = 0; i < RF02_TOTAL_CARTAS; i++) {
        simbolos[i] = originais[i];
    }
    for (int i = RF02_TOTAL_CARTAS - 1; i > 0; i--) {
        int j = rand() % (i + 1);
        const char *aux = simbolos[i];
        simbolos[i] = simbolos[j];
        simbolos[j] = aux;
    }
    iniciada = rf02_inicializar(&jogo, simbolos);
    IniciarTurnos();
    IniciarPontuacao();
}
```

RF02 Escolha e comparação de cartas

```
ResultadoJogada rf02_escolher_carta(JogoMemoria *jogo, int posicao) {
    Carta *primeira;
    Carta *segunda;

    if (jogo == 0 || !posicao_valida(posicao)) {
        return JOGADA_INVALIDA;
    }

    if (jogo->jogada.aguardando) {
        return AGUARDANDO_LIBERACAO;
    }

    if (jogo->tabuleiro.cartas[posicao].estado != CARTA_OCULTA) {
        return JOGADA_INVALIDA;
    }

    if (jogo->jogada.primeira == -1) {
        jogo->jogada.primeira = posicao;
        jogo->tabuleiro.cartas[posicao].estado = CARTA_VISIVEL;
        return PRIMEIRA_CARTA;
    }

    jogo->jogada.segunda = posicao;
    primeira = &jogo->tabuleiro.cartas[jogo->jogada.primeira];
    segunda = &jogo->tabuleiro.cartas[posicao];
    segunda->estado = CARTA_VISIVEL;

    if (simbolos_iguais(primeira->simbolo, segunda->simbolo)) {
        primeira->estado = CARTA_ENCONTRADA;
        segunda->estado = CARTA_ENCONTRADA;
        limpar_jogada(&jogo->jogada);
        return PAR_ENCONTRADO;
    }

    jogo->jogada.aguardando = 1;
    return PAR_DIFERENTE;
}
```

RF03 Atualização dos turnos

```
int rf03_processar_resultado(ControleTurnos *turnos, int resultado) {
    if (!controle_valido(turnos)) {
        return 0;
    }

    if (resultado == PAR_ENCONTRADO || resultado == PAR_DIFERENTE) {
        turnos->ultimoJogador = turnos->jogadorAtual;
        if (resultado == PAR_DIFERENTE) {
            turnos->jogadorAtual = 3 - turnos->jogadorAtual;
        }
    } else if (resultado != JOGADA_INVALIDA && resultado != PRIMEIRA_CARTA &&
               resultado != AGUARDANDO_LIBERACAO) {
        return 0;
    }

    return turnos->jogadorAtual;
}
```

RF04 Registro de pontos e tentativas

```
int rf04_registrar(Placar *p, int jogador, int resultado) {
    if (!p || jogador < 1 || jogador > 2 || p->paresEncontrados >= 8 ||
        (resultado != PAR_ENCONTRADO && resultado != PAR_DIFERENTE)) {
        return 0;
    }
    p->jogadores[jogador - 1].tentativas++;
    p->totalTentativas++;
    if (resultado == PAR_ENCONTRADO) {
        p->jogadores[jogador - 1].pontos++;
        p->paresEncontrados++;
    }
    return 1;
}
```

RF05 Verificação do resultado

```
int rf05_apurar_placar(const JogoMemoria *jogo,
                       const Placar *placar)
{
    if (placar == 0) return RESULTADO_INVALIDO;

    int p1 = placar->jogadores[0].pontos;
    int p2 = placar->jogadores[1].pontos;

    if (verificarFimJogo(p1, p2) < 0 ||
        placar->paresEncontrados != p1 + p2 ||
        placar->jogadores[0].tentativas < p1 ||
        placar->jogadores[1].tentativas < p2 ||
        placar->totalTentativas < 0 ||
        (long long)placar->totalTentativas !=
            (long long)placar->jogadores[0].tentativas +
            placar->jogadores[1].tentativas) {
        return RESULTADO_INVALIDO;
    }

    return rf05_apurar_vencedor(jogo, p1, p2);
}
```

Compilação e execução

O script `build.bat` compila os cinco módulos em C com Emscripten e gera os arquivos WebAssembly e JavaScript de suporte na pasta `docs`. A lógica do jogo é executada no próprio navegador.

Para compilar, é necessário ativar o ambiente do Emscripten e executar, na raiz do projeto:

```
None
build.bat
```

Para disponibilizar os arquivos localmente, utiliza-se:

```
None
python -m http.server 8000
```

Com o servidor HTTP local em execução, o protótipo visual pode ser acessado em `http://localhost:8000/docs/`. A interface encontra-se em desenvolvimento; a demonstração integrada dos cinco requisitos funcionais é realizada pela página de testes.

Verificação realizada

Os testes de integração estão implementados em `tests/teste_integracao.c`. Para compilá-los com Emscripten, executa-se:

```
None
testar.bat
```

Com o servidor HTTP local em execução na raiz do projeto, os testes podem ser acessados em `http://localhost:8000/tests/teste_integracao.html`.

A execução no navegador apresentou a mensagem: “RF01 + RF02 + RF03 + RF04 + RF05: todos os testes passaram.”

Foram verificados a inicialização do tabuleiro, o processamento das jogadas, a validação dos pares, a manutenção da vez no acerto, a troca de jogador no erro, a pontuação e a contagem de tentativas. Também foram testados jogadas inválidas, vitória de cada jogador, empate, bloqueio de jogadas após o encerramento e reinicialização da partida.

Esses testes validam a integração da lógica em C compilada para WebAssembly. A integração completa da interface visual e sua avaliação de acessibilidade permanecem como etapas posteriores.

Critérios de aceitação

Uma partida não iniciada não é considerada concluída. Enquanto houver pares pendentes, a partida continua. A pontuação e as tentativas da última jogada são atualizadas antes da apuração do vencedor. Após o encerramento, novas jogadas não alteram o resultado. Reiniciar limpa o estado das cartas, os pontos, as tentativas e os turnos.

Conclusão

O projeto reúne inicialização do tabuleiro, validação de pares, alternância de turnos, pontuação e encerramento em uma versão integrada em C. Os testes automatizados verificaram as regras e a execução dos arquivos compilados. A equipe deve utilizar esta versão consolidada na revisão final e no ensaio da apresentação, antes da publicação e do envio do PDF no Moodle.

Referências

EMSCRIPTEN. Interacting with code. https://emscripten.org/docs/porting/connecting_cpp_and_javascript/Interacting-with-code.html. Acesso em 15 out. 2026.

EMSCRIPTEN. Emscripten Compiler Frontend emcc. https://emscripten.org/docs/tools_reference/emcc.html. Acesso em 15 out. 2026.

EQUIPE DO PROJETO. Jogo da Memória. Repositório base: https://github.com/menezzs/jogo-memoria/tree/Teste. Versão base consultada em 15 out. 2026, commit 177550696bd75cc88afa7634390f48cd1c2cbd1c. A versão consolidada acompanha o pacote entregue.